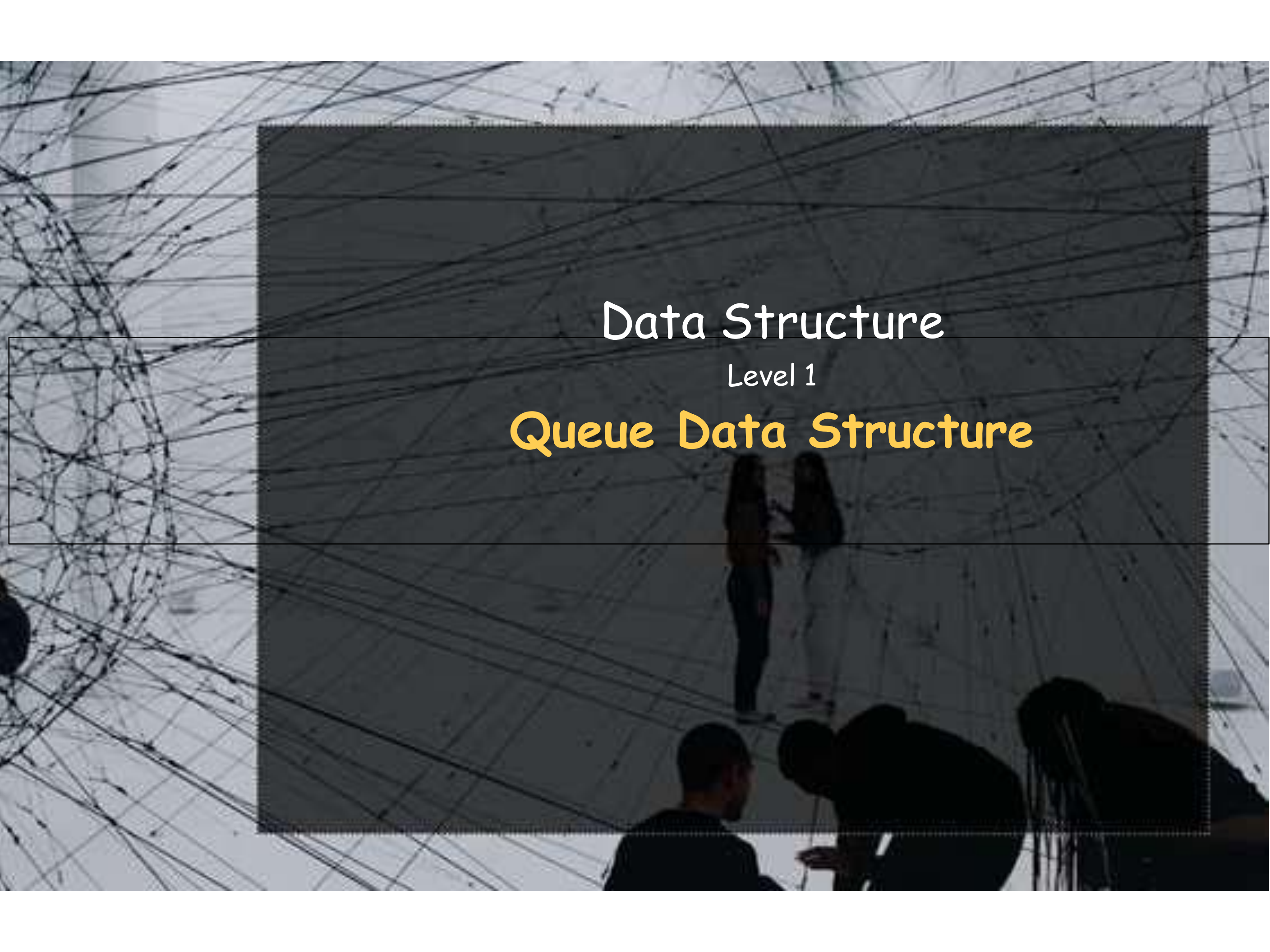


Data Structure Level 1

Queue Data Structures

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in motion, which adds a sense of activity and human interaction to the technical theme.

Data Structure

Level 1

Queue Data Structure

Definition

A Queue is a data structure where elements are inserted from the rear (back) and removed from the front.

STL : Standard Template Library
We take stack from it

```
#include <queue>
```

```
queue<int> MyQueue;
```

Explanation

- ◆ Elements are added using `push()` and removed using `pop()`.
- ◆ `front()` accesses the first element, and `back()` accesses the last element.
- ◆ Queues are widely used in scheduling tasks, buffering, and managing requests..

Common Queue Operations

-  `push(x)` → Add element to rear
-  `pop()` → Remove element from front
-  `front()` → Access first element
-  `back()` → Access last element
-  `empty()` → Check if queue is empty
-  `size()` → Number of elements

Procedures (Program Flow)

- 1 Include required libraries (<queue>, <iostream>)
- 2 Declare a queue → `queue<int> MyQueue;`
- 3 Add elements using `push()`
- 4 Print size → `MyQueue.size()`
- 5 Access front and back → `front()` and `back()`
- 6 Loop until empty → print and remove elements using `pop()`

Example

Queue after pushing 10, 20, 30, 40:

→ Front: 10

→ Back: 40

Popping elements results in: 10, 20, 30, 40



Comparison

■ Queue

- ✓ FIFO order
- ✓ Insert from rear
- ✓ Remove from front
- ✓ Used in scheduling and buffering

■ Stack

- ✓ LIFO order
- ✓ Insert and remove from top
- ✓ Used in recursion and undo operations



Note

Queue does not allow random access like arrays or vectors; you can only access front and back elements.

Warning

Calling `front()` or `pop()` on an empty queue may cause runtime errors.

Summary

- ✓ Queue follows FIFO principle
- ✓ Supports insertion at rear and deletion from front
- ✓ Access to front and back elements only
- ✓ Useful for scheduling, buffering, and task processing

Thank you 😊